

Recognizing Niche Animal Behaviors with Transfer Learning on Consumer Hardware: A Rabbit Activity Monitor

Joe Patricio

<https://bunny-tracker.app>

June 2026

Abstract

I built a self-hosted system that recognizes rabbit behaviors from a live camera feed and emails the owner when something noteworthy happens. The five behaviors are grooming, standing, yawning, zoomies (bursts of rapid play movement), and normal (resting). The technical core is a transfer-learning classifier that adapts a general-purpose image model, MobileNet V2 pre-trained on ImageNet, to a narrow domain with no public dataset: the spontaneous behaviors of one pet rabbit at home. Training a deep network from scratch is not an option with only 173 hand-labeled clips, so I freeze the convolutional backbone, treat it as a fixed 1280-dimensional feature extractor, and train a small dense classifier on top of it. The model trains in the browser in under a minute and runs in real time on a consumer webcam and a desktop CPU, with no cloud ML services. I describe the architecture, the data-collection and labeling workflow, the training protocol, and the evaluation method (held-out validation accuracy and a per-class confusion matrix), and I report roughly 81% validation accuracy across five classes from a single rabbit. The paper closes with the limits of small-sample, single-subject behavior modeling and a plan to improve robustness.

1 Introduction

Commercial pet cameras stream video and, at best, flag generic motion. None tell you what the animal is doing. For a prey species like a rabbit, behavior carries real health and welfare signal. Zoomies mark a happy, energetic animal. A drop in grooming or movement can be an early warning of illness. Classify these behaviors and surface clips of the interesting ones, and a passive camera becomes a behavioral monitor.

The hard part is data scarcity in a niche domain. No ImageNet exists for rabbit behavior, and one hobbyist cannot hand-label the tens of thousands of examples a convolutional network needs to learn visual features from raw pixels. Transfer learning fits this regime well. A network trained on a large general dataset has already learned reusable low- and mid-level features such as edges, textures, shapes, and body parts, so only the final decision layer has to be re-learned for the new task.

This paper contributes three things:

1. A transfer-learning pipeline that adapts MobileNet V2 to five rabbit behaviors using 173 labeled clips, trainable end to end in a web browser.

2. A frame-averaging embedding scheme that turns short video clips into single fixed-length feature vectors, letting a static image model handle short-horizon behavior classification without a temporal network.
3. A complete self-hosted real-time system, covering capture, inference, alerting, and a labeling/retraining loop, that runs on consumer hardware with no cloud dependency. The system shows that niche behavior recognition does not require specialized infrastructure.

2 Related Work and Background

Transfer learning. Reusing a network pre-trained on a large dataset as a feature extractor for a smaller task is standard practice [3, 2]. The early layers of a vision model learn generic features that carry across domains, so adapting to a new task often means retraining only the final classifier. This cuts both the data and the compute the task demands.

MobileNet V2. MobileNet V2 [1] is a convolutional architecture built for efficiency on mobile and edge devices. It uses depthwise-separable convolutions and inverted residual blocks to reach competitive accuracy at a fraction of the parameter count of larger networks. At width multiplier $\alpha = 1.0$ it produces a 1280-dimensional embedding per input image. I use it only as a frozen feature extractor.

Behavior recognition from video. Full video-action models such as 3D convolutions, two-stream networks, and temporal transformers capture motion dynamics, but they need large labeled datasets and heavy compute. A single-subject, low-data home setting rules them out. My approach trades temporal modeling for tractability: I summarize a short clip by averaging its per-frame embeddings, which captures the dominant visual content of the behavior and stays trainable on 173 examples.

3 System Overview

The system has four cooperating parts:

```
[camera] -> capture agent (ffmpeg) -> MobileNet V2 embedding -> classifier -> behavior
      |                                     |
      +--> rolling video clips              +--> email alert + clip
      +--> live stream -> dashboard          +--> public text feed
```

- **Capture agent.** A headless service reads the camera with a single `ffmpeg` process, samples a frame every few seconds for inference, and records short rolling clips.
- **Classifier.** The transfer-learning model this paper describes. The agent loads it and runs it on each sampled frame.
- **Web application.** Three authenticated tools: a live camera view, a Label Studio for tagging clips, and a Training Studio that extracts features and trains the classifier in the browser.
- **Alerting and review loop.** Noteworthy behavior triggers an email with the clip. Captured clips flow back into Label Studio, which closes a human-in-the-loop retraining cycle.

This paper focuses on the classifier (Sections 4–7). I describe the surrounding system only where it bears on data collection and deployment.

4 Dataset

4.1 Collection

Clips come from two sources: recordings the system’s own camera captured of my pet rabbit, and supplementary short clips that broaden coverage of the rarer behaviors. Each clip runs a few seconds at 640×360 . One rabbit in one room makes the dataset single-subject and single-scene, which matters for generalization (Section 8).

4.2 Labeling

I labeled clips through a custom Label Studio interface with one-key shortcuts (Z/Y/N/G/S) and an auto-advancing filmstrip for fast manual annotation. Labels write atomically, using serialized writes with a temp-file rename plus daily backups, so concurrent edits cannot corrupt them. I added that safeguard after losing labels to a race condition during development.

4.3 Composition

The labeled set behind the reported model holds 173 clips across five classes:

Class	Clips	Share	Description
normal	55	31.8%	Resting / no notable activity
zoomies	34	19.7%	Rapid bursts of running and binkying
grooming	31	17.9%	Self-cleaning, licking, face-wiping
yawn	29	16.8%	Open-mouth yawn
standing	24	13.9%	Upright on hind legs
Total	173	100%	

The distribution tilts toward *normal*, the most common state, and *standing* is the rarest. Every class has enough examples to train on, but the imbalance shows up in per-class performance (Section 7).

5 Methodology

5.1 Clip-to-vector representation

A static image classifier cannot read a video clip. I turn each clip into one feature vector in three steps:

1. **Frame sampling.** Take `FRAMES_PER_CLIP` = 8 evenly spaced frames across the clip (up to 95% of its duration), each resized to 224×224 .
2. **Embedding.** Pass each frame through frozen MobileNet V2 ($\alpha = 1.0$) and read the penultimate-layer activation, a 1280-dimensional embedding.
3. **Temporal pooling.** Average the eight frame embeddings into one 1280-vector for the clip.

Averaging is a chosen simplification. It drops fine temporal ordering but keeps the behavior’s dominant visual signature, such as an upright silhouette for *standing* or an open mouth for *yawn*.

That signal separates the classes, and averaging keeps the input fixed-length so the model trains on a small dataset.

5.2 Classifier architecture

On top of the frozen 1280-dim features I train a small fully-connected network:

```
Input: 1280-d embedding
-> Dense(256, ReLU, L2=1e-4)
    -> Dropout(0.4)
        -> Dense(128, ReLU, L2=1e-4)
            -> Dropout(0.3)
                -> Dense(5, Softmax)
Output: probability distribution over 5 classes
```

The two dropout layers (0.4, 0.3) and the L2 weight penalty (10^{-4}) target the main risk: overfitting on 173 samples with a 1280-dimensional input. The head holds roughly 360K parameters, and the serialized weights come to about 1.4 MB.

5.3 Training protocol

Hyperparameter	Value
Optimizer	Adam
Learning rate	0.001
Loss	Categorical cross-entropy
Epochs	40
Batch size	16
Train/validation	80% / 20% (shuffled split)
Frames per clip	8 (averaged)
Backbone	MobileNet V2, $\alpha = 1.0$ (frozen)

Training runs in the browser with TensorFlow.js. The frozen backbone means the optimizer touches only the dense head, so a 40-epoch run finishes in under a minute on a desktop CPU. I serialize the trained model and POST it to the server, where the capture agent loads it for live inference.

5.4 Inference

At run time the agent samples one frame every three seconds, embeds it with the same frozen MobileNet V2, and runs the dense classifier to get a label and a confidence (the softmax probability). Training and inference differ in one respect: training averages eight frames per clip, while live inference classifies single frames for lower latency. Section 8 returns to this mismatch.

6 Evaluation Methodology

I evaluate the classifier with two measures, both computed on the held-out 20% validation split the model never trains on:

1. **Validation accuracy.** The fraction of validation clips whose predicted class matches my label. This is the headline number.
2. **Confusion matrix.** A 5×5 table of true class (rows) against predicted class (columns) that shows which classes the model confuses. For an imbalanced multi-class problem this beats accuracy alone, because it reveals whether errors spread evenly or cluster between two look-alike behaviors.

The Training Studio computes both at the end of every run, so evaluation repeats on demand and stays visible in the dashboard rather than living in a one-off offline script. I also track training accuracy per epoch to watch convergence and catch overfitting, which shows up as a wide gap between train and validation accuracy.

7 Results

7.1 Headline accuracy

The reported model reaches about 81% validation accuracy across the five classes. Random guessing on a five-way problem lands near 20%, and a majority-class baseline that always predicts *normal* reaches about 32%. The classifier clears both, so the transferred features carry genuine discriminative signal for rabbit behavior.

7.2 Per-class behavior (confusion matrix)

The Training Studio regenerates the confusion matrix at the end of each run (Training Studio → Results). The error structure follows a few intuitive patterns:

- **High-motion, distinct shapes** (*zoomies*, *standing*) separate best. A running or upright rabbit has a silhouette unlike a resting one.
- **Low-motion, similar postures** (*grooming* versus *normal*) confuse most. A grooming rabbit and a resting rabbit fill nearly identical frames, and the paw wipe that distinguishes them rarely lands in a sampled frame.
- **yawn** is brief. The open-mouth frame is a small slice of the clip, so averaging dilutes its signature and sometimes pulls it toward *normal*.

Exact per-class counts depend on the shuffled split and appear in the dashboard at training time. The schema below is ready to populate from your latest run before publication.

true ↓ \ pred →	grooming	normal	standing	yawn	zoomies
grooming	—	—	—	—	—
normal	—	—	—	—	—
standing	—	—	—	—	—
yawn	—	—	—	—	—
zoomies	—	—	—	—	—

7.3 Efficiency

The system hits its real-time, consumer-hardware target. Inference runs on CPU through TensorFlow.js at a 3-second sampling cadence, the model weighs about 1.4 MB, and a full retrain finishes in under a minute in the browser. No GPU and no cloud ML service enter the pipeline at any stage.

8 Discussion and Limitations

The results support the thesis that transfer learning makes niche, data-scarce behavior recognition tractable on commodity hardware. They also leave clear room to improve, and I list the limits as the agenda for future work:

1. **Single-subject, single-scene data.** All training data shows one rabbit in one room. The model has probably learned scene-specific cues alongside behavioral ones, so I would not expect it to transfer to another rabbit or another environment without more data.
2. **Small, imbalanced dataset.** With 173 clips the validation split holds about 35, so per-class metrics swing widely and the headline accuracy carries a wide confidence interval.
3. **Train/inference representation mismatch.** Training averages eight frames per clip; live inference classifies single frames. A single frame is noisier and carries less information than an averaged clip, which lowers live confidence relative to validation accuracy. The effect hits transient (*yawn*) and motion-defined (*zoomies*) behaviors hardest, since their signature may not appear in an arbitrary frame.
4. **No explicit temporal modeling.** Frame averaging cannot tell apart behaviors that differ only in motion, such as the act of grooming versus a still rabbit. This drives the largest source of class confusion.
5. **Closed-world assumption.** The model assigns one of five rabbit-behavior labels to every frame, including frames with humans or an empty room, so it returns confident but meaningless predictions in those cases.

9 Future Work

- **Temporal modeling.** Swap frame averaging for a small sequence model over the per-frame embeddings, such as an LSTM or a light temporal transformer, to capture motion and attack the grooming/normal and yawn confusions.
- **Align training and inference.** Run inference over a short rolling window of frames and average their embeddings, matching the training representation, to raise live confidence.
- **Grow and balance the dataset.** Add more subjects, rooms, and lighting conditions, including low light and night, and rebalance the under-represented classes. The human-in-the-loop labeling workflow already collects this data as the system runs.
- **Add an “other/absent” class.** A negative class for humans and empty scenes addresses the closed-world limit.
- **Calibrate confidence.** Calibrate the softmax probabilities so they become reliable enough to gate alerts on their own.

10 Conclusion

A niche, data-scarce animal-behavior task, classifying five rabbit behaviors, reaches about 81% validation accuracy with transfer learning on a frozen MobileNet V2 backbone and a small dense head, trained on 173 hand-labeled clips and running in real time on consumer hardware. Treating a general-purpose image model as a fixed feature extractor and summarizing short clips as averaged embeddings sidesteps the data and compute that training from scratch or a full video-action network would demand. The system also embeds a human-in-the-loop labeling and retraining loop, which gives a practical path to improve the model as more data arrives. The honest limits, single-subject data, a small validation set, and no temporal modeling, set a concrete and reachable agenda for the next round.

References

- [1] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen. *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. CVPR, 2018.
- [2] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson. *How transferable are features in deep neural networks?* NeurIPS, 2014.
- [3] S. J. Pan and Q. Yang. *A Survey on Transfer Learning*. IEEE Transactions on Knowledge and Data Engineering, 2010.
- [4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. JMLR, 2014.
- [5] D. P. Kingma and J. Ba. *Adam: A Method for Stochastic Optimization*. ICLR, 2015.
- [6] D. Smilkov et al. *TensorFlow.js: Machine Learning for the Web and Beyond*. MLSys, 2019.
- [7] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. *ImageNet: A Large-Scale Hierarchical Image Database*. CVPR, 2009.